DEVSPACE

PLATFORM-AGNOSTIC · SERVERLESS · OPEN SOURCE · ZERO LOCK-IN

SERVERLESS FIRST | ANY CLOUD | APACHE 2.0

---

01 USER FLOW

$ devspace login → OIDC Auth → JWT Token → $ devspace up → Lambda → Terraform → EB

02 CLIENT LAYER

⌨️ CLI Tool

Single binary. Go or Python. Cross platform. The only thing users install.

- devspace login
- devspace up
- devspace ssh
- devspace down
- devspace ls
- --type container|vm|gpu|metal

🌐 Browser UI

Static files on S3 + CloudFront. Private ALB. Xterm.js for in-browser terminal. VS Code web for IDE.

- S3 Static Hosting
- CloudFront CDN
- Private ALB
- Xterm.js Terminal
- WebSocket API

03 AUTH & API GATEWAY LAYER

🔐 OIDC Auth

Plug into your existing identity provider. No custom auth logic. JWT tokens validated on every request.

- Your OIDC Provider
- JWT Validation
- Lambda Authorizer
- Per-request auth

🚪 API Gateway

REST API for workspace management. WebSocket API for terminal streaming. All protected by Lambda authorizer.

REST API
WebSocket API
Lambda Authorizer
Rate limiting
VPC private endpoint

04 CONTROL PLANE — LAMBDA FUNCTIONS

⚡ workspace_create

Calls Terraform to provision EBS volume + compute. Stores state in DynamoDB.

- Terraform apply
- EBS provision
- DynamoDB write

💤 workspace_sleep

Stops Fargate task or EC2. EBS volume persists. Cost drops to zero. Triggered after idle timeout.

- Stop compute
- Keep volume
- Cost = $0

🌅 workspace_wake

Restarts Fargate task or EC2. Re-attaches same EBS volume. Ready in 10-30 seconds.

- Start compute
- Re-attach EBS
- ~15s cold start

🔗 workspace_connect

Returns SSM session token or Wireguard config for SSH access. Validates user owns workspace.

- SSM token
- Wireguard config
- Ownership check

🗑️ workspace_destroy

Terraform destroy. Optionally keeps EBS volume as backup. Cleans up all resources.

- Terraform destroy
- Optional EBS keep
- DynamoDB cleanup

👁️ idle_monitor

EventBridge scheduled Lambda. Checks last activity. Auto-sleeps workspaces idle over 10 mins.

- EventBridge
- Activity check
- Auto-sleep

05 PROVIDER ABSTRACTION LAYER — KEY DIFFERENTIATOR

🔌 WorkspaceProvider Interface

Every cloud implements the same interface. Swap providers without touching CLI or API

code.

```
# core/provider.py class WorkspaceProvider(ABC): create_workspace(user_id, template, type) → Workspace destroy_workspace(workspace_id) → bool sleep_workspace(workspace_id) → bool wake_workspace(workspace_id) → bool get_connection(workspace_id) → Connection get_status(workspace_id) → Status
```

☁️ AWS Provider

Fargate / EC2
EBS Volumes
SSM Tunneling
DynamoDB State
S3 TF State

☁️ GCP Provider

Cloud Run Jobs
Persistent Disks
IAP Tunneling
Firestore State
GCS TF State

☁️ Azure Provider

Container Apps
Managed Disks
Bastion Host
CosmosDB State
Blob TF State

🏠 K8s Provider

vCluster
PVC Volumes
Wireguard
etcd State
Local TF State

💻 Local Provider

Docker / k3s
Bind Mounts
Direct SSH
SQLite State
Local TF State

➕ Community

Hetzner
OVH Bare Metal
DigitalOcean
Custom infra
Open interface

06 WORKSPACE TYPES

⚡ Container

~$0.01/hr

Fargate task
EBS gp3 20GB
SSM tunnel
Shared node
Best for: light dev

🖥️ VM

~$0.04/hr

EC2 t3.medium
EBS gp3 50GB
SSM tunnel
Dedicated core
Best for: power dev

🤖 GPU

~$0.50/hr

EC2 g4dn
EBS gp3 100GB
SSM tunnel
NVIDIA T4
Best for: ML/AI

⚙️ Bare Metal

~$0.05/hr

Hetzner / OVH
Local NVMe
Wireguard
Full server
Best for: enterprise

07 STATE MANAGEMENT

TERRAFORM STATE (S3)

- → EBS volumes per user
- → EC2 / Fargate task defs
- → Security groups
- → IAM roles
- → Network config
- → Volume attachments

Changes on: provision / destroy only

RUNTIME STATE (DYNAMODB)

- → Workspace status (running/sleeping)
- → User → workspace mapping
- → Volume ID + AZ per user
- → Last activity timestamp
- → Connection details
- → Template used

Changes on: every user action

08 NETWORKING & SSH ACCESS

🔒 AWS — SSM Session Manager

No open ports. No public IPs. AWS handles tunneling. Free. Works inside VPC.

```
# User SSH via SSM aws ssm start-session \ --target ecs:cluster/task-id \ --document-name AWS-StartSSHSession
```

🔐 Other Clouds — Wireguard

Wireguard sidecar in every workspace container. Peer-to-peer encrypted tunnel. Platform agnostic.

```
# Wireguard sidecar config sidecar: wireguard port: 51820 peer: user-public-key allowed_ips: 10.0.0.2/32
```

09 IDE SUPPORT

💻 VS Code Remote SSH

Zero engineering. Users point VS Code at workspace SSH. Full IDE experience.

- Zero build effort
- Full VS Code
- Extensions work

🌐 Browser Terminal

Xterm.js in S3 UI. WebSocket → API Gateway → Lambda → SSH proxy into workspace.

- Xterm.js
- WebSocket API
- No local install

🧠 JetBrains Gateway

JetBrains Gateway plugin. SSH into workspace. Full IntelliJ/PyCharm remotely.

- Zero build effort
- All JetBrains IDEs
- SSH native

10 SECURITY MODEL

🛡️ Isolation

Each workspace in its own security group. No cross-workspace traffic. VPC private subnets only.

- Per-user security group
- Private subnets
- No public IPs
- NACLs

🔑 Auth Chain

OIDC → JWT → Lambda Authorizer → Ownership check → Workspace access. Trust nothing from frontend.

- OIDC validated
- JWT per request
- Ownership enforced
- CSP headers

11 BUILD TIMELINE

📅 Week 1-2 — Core AWS

VPC, IAM, DynamoDB schema, Terraform workspace module, EBS + Fargate task, Lambda functions.

📅 Week 3 — API + Auth

API Gateway REST + WebSocket, Lambda authorizer, OIDC integration, SSM tunneling.

📅 Week 4 — CLI + UI

CLI binary (Go/Python), S3 + CloudFront + ALB setup, Xterm.js terminal, workspace dashboard.

📅 Week 5-6 — Polish

Auto-sleep Lambda, provider abstraction layer, VS Code SSH integration, testing, first real user.

12 DEVSPACE VS CODER

❌ CODER	✅ DEVSPACE
Single replica limit (OSS)	Infinitely scalable serverless
Enterprise license gates	100% open, zero gates
AWS only friendly	Any cloud or on-prem
Servers always running	Auto-sleep to zero cost
Pay flat pricing always	Pay per active minute only

AGPL license restrictions	Apache 2.0 — do anything
Heavy UI required	CLI first, UI optional
Poor support at high cost	Self sufficient, own it fully